



APPLIED DATA SCIENCE CAPSTONE

Predicting Falcon 9 First-Stage Landing Success

A data science analysis of SpaceX launch outcomes — from data collection through predictive modeling

Abdou A. Diakite

IBM Data Analyst Certificate

August 11, 2026

Github:



BACKGROUND

Introduction

Project Background

- SpaceX advertises Falcon 9 launches at ~\$62M, far below competitors' ~\$165M+, largely because SpaceX reuses the first stage instead of discarding it.
- Reuse is only possible when the first stage successfully lands, so landing success directly determines launch cost.
- A competing company bidding against SpaceX would benefit from estimating this cost — which requires predicting landing success.

Problem Statement

Can we predict whether the Falcon 9 first stage will land successfully, using publicly available launch data — payload mass, orbit type, launch site, booster version, and more?

Key Questions Explored

- Which variables most influence landing outcome?
- How has success rate changed over time?
- Which classification model predicts outcome best?



OVERVIEW

Executive Summary

- This project aims to predict whether the Falcon 9 first stage will land successfully so that a competing company can make more informed launch bids.
- Methods used: data collection (API + web scraping), data wrangling, exploratory data analysis, interactive visual analytics and predictive classification modeling.
- After comparing multiple models, we find that the Falcon 9 booster has a high probability of successful landing (87.7% based on decision tree classifier). Success is highly correlated with launch site, orbits, boosters and payload mass.
- Success rate has reached 90% in recent years, thanks to new generation of boosters.

87.7%

*Best model accuracy
Decision Tree classifier*

90

*Launch records analyzed
Falcon 9 launches (2010–2020)*

66.7%

*Overall landing success rate
60 of 90 landings succeeded*



```

graph LR
    1[1 GET request to api.spacexdata.com/v4/launches/past] --> 2[2 Parse JSON response into a pandas DataFrame]
    2 --> 3[3 Use rocket, payload, launchpad, core IDs to make follow-up GET requests for details]
    3 --> 4[4 Merge nested results into flat columns per launch]
    4 --> 5[5 Filter to Falcon 9 launches only (drop Falcon 1)]
  
```

1 GET request to `api.spacexdata.com/v4/launches/past`

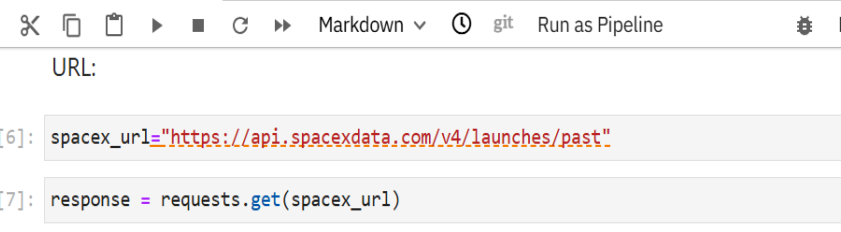
2 Parse JSON response into a pandas DataFrame

3 Use rocket, payload, launchpad, core IDs to make follow-up GET requests for details

4 Merge nested results into flat columns per launch

5 Filter to Falcon 9 launches only (drop Falcon 1)

- `requests.get(url).json()` against the SpaceX v4 API
- IDs returned per launch (rocket, payloads, launchpad, cores) require separate GET calls to resolve into readable fields
- Custom functions (e.g. `getBoosterVersion()`, `getLaunchSite()`) map IDs → names and append to lists
- Combine into a single DataFrame; export to `spacex_api.csv`



The screenshot shows a JupyterLab window with a single notebook. The notebook has a title bar with a close button and a '+' icon. Below the title bar is a toolbar with icons for saving, adding, deleting, and running code, as well as buttons for 'Markdown', 'git', and 'Run as Pipeline'. The main area of the notebook contains three cells:

- A text cell with the label 'URL:'.
- A code cell with the following Python code:


```
[6]: spacex_url="https://api.spacexdata.com/v4/launches/past"
```
- A code cell with the following Python code:


```
[7]: response = requests.get(spacex_url)
```

Below the code cells is a text cell with the instruction 'Check the content of the response'.

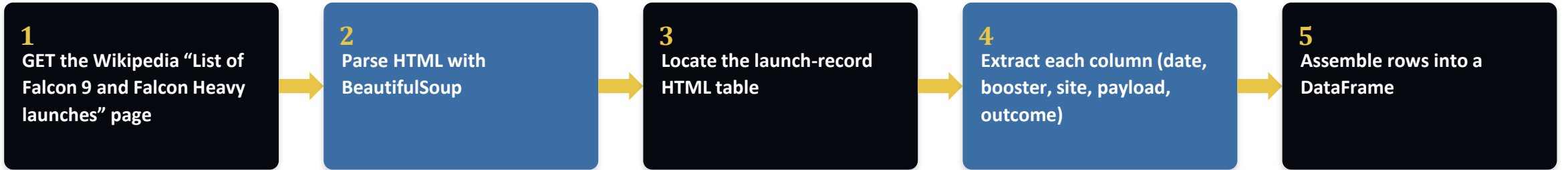
At the bottom of the screenshot, the output of the code is visible, showing the start of an HTML document:

```
[8]: print(response.content)
b'<!DOCTYPE html>\n<!--[if lt IE 7]> <html class="no-js ie6 oldie" lang="en-US"> <![endif]-->\n<!--[if IE 7]> <html class="no-js ie7 oldie" lang="en-US"> <![endif]-->\n<!--[if IE 8]> <html class="no-js ie8 oldie" lang="en-US"> <![endif]-->\n<!--[if gt IE 8]><!--> <html class="no-js" lang="en-US"> <!--<![endif]-->\n<head>\n<title>spacexdata.com | 525: SSL handshake failed</title>\n<meta charset="UTF-8" />\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />\n<meta http-equiv
```

Notebook / Code: github.com/your-username/your-repo-name



Web Scraping (Wikipedia)



Key Steps

- `requests.get()` the Wikipedia revision URL, then `BeautifulSoup(html, 'html.parser')`
- `soup.find_all('table', 'wikitable plainrowheaders collapsible')` to locate the launch table
- Custom parsing functions per column (date/time, booster version, payload mass, orbit, outcome)
- Append parsed cells into a dict of lists → `pd.DataFrame`; export to `spacex_web_scraped.csv`

```
[11]: # use requests.get() method with the provided static_url and headers
# assign the response to a object
response = requests.get(static_url, headers=headers)

Create a BeautifulSoup object from the HTML response

[12]: soup = BeautifulSoup(response.text, "html.parser")

Print the page title to verify if the BeautifulSoup object was created properly

[13]: print(soup.title)

<title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>

TASK 2: Extract all column/variable names from the HTML table header

Next, we want to collect all relevant column names from the HTML table header

Let's try to find all tables on the wiki page first. If you need to refresh your memory about BeautifulSoup, please check the external
reference link towards the end of this lab

[16]: # Use the find_all function in the BeautifulSoup object, with element type 'table'.
# Assign the result to a list called 'html_tables'
html_tables = soup.find_all("table")

Starting from the third table is our target table contains the actual launch records.

[15]: # Let's print the third table and check its content
third_launch_table = html_tables[2]
print(third_launch_table)

<table class="wikitable plainrowheaders collapsible sticky-header" id="2026ytd" style="width: 100%;">
<tbody id="mGGk"><tr id="mGGk"><th id="mGGs" scope="col">Flight No.</th>
<th id="mGGv" scope="col">Date and<br id="mGGb">time (a href="https://en.wikipedia.org/wiki/Coordinated_Universal_Time" id="mGG4" rel="mw:
Wikilink" title="Coordinated Universal Time">UTC</a></th>
<th id="mGGb" scope="col">a href="https://en.wikipedia.org/wiki/List_of_Falcon_9_first-stage_boosters" id="mGG4" rel="mw:Wikilink" title="Li
st of Falcon 9 first-stage boosters">Version,<br id="mGHE">booster</a><sup about="#mw2595" class="mw-ref reference" data-mw="{"name":"re
f","atts":{"name":"booster","group":"lower-alpha"},"body":{"html":""},"parts":[{"template":{"target":{"wt":"efn","href":"./Template:Ef
ams":{"name":{"wt":"booster"},"i":0}}}} id="cite_ref-booster_37-1" rel="dc:references" typeof="mw:Transclusion mw:Extension/ref">a data-mw-r
```

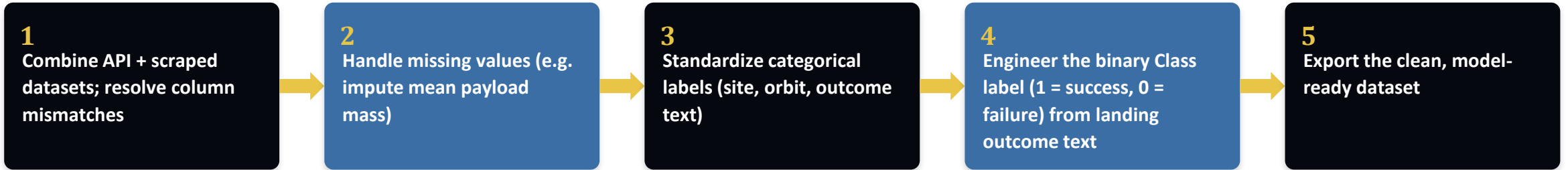


Notebook / Code: github.com/your-username/your-repo-name



PREPARATION

Data Wrangling Methodology



What Counts as a “Successful Landing”

- Outcome strings like 'True ASDS', 'True RTLS', 'True Ocean' → Class = 1
- 'False ASDS', 'False RTLS', 'None None' → Class = 0
- This binary label becomes the target variable for the classification models

```
labs-jupyter-spaces-Data win: X +
TASK 2: Calculate the number and occurrence of each orbit
Use the method .value_counts() to determine the number and occurrence of each orbit in the column Orbit
Note: Do not count GTO, as it is a transfer orbit and not itself geostationary.

[9]: # Apply value_counts on Orbit column
df["Orbit"].value_counts()

[9]: Orbit
GTO    27
ISS    21
VLEO   14
PO      9
LEO      7
SSO      5
MEO      3
ES-L1    1
HEO      1
SO        1
GEO      1
Name: count, dtype: int64

TASK 3: Calculate the number and occurrence of mission outcome of the orbits
Use the method .value_counts() on the column Outcome to determine the number of landing_outcomes. Then assign it to a variable landing_outcomes.

[13]: # Landing_outcomes = values on Outcome column
landing_outcomes = df["Outcome"].value_counts()
landing_outcomes

[13]: <bound method IndexOpsMixin.value_counts of 0      None None
1      None None
2      None None
3      False Ocean
4      None None
...
85     True ASDS
86     True ASDS
87     True ASDS
88     True ASDS
```



Notebook / Code: github.com/your-username/your-repo-name



EXPLORATORY ANALYSIS

EDA with Data Visualization

Charts Used & Purpose

Scatter — Flight Number vs. Launch Site

Reveal whether success improved over successive flights, per site

Scatter — Payload Mass vs. Launch Site

Check whether heavier payloads cluster at certain sites or fail more

Bar — Success Rate by Orbit Type

Compare landing reliability across orbit classes (LEO, GTO, ISS, ...)

Scatter — Flight Number / Payload vs. Orbit Type

Look for interaction effects between orbit, payload, and experience

Line — Yearly Launch Success Trend

Show how success rate evolved as SpaceX iterated on booster design



Notebook / Code: github.com/your-username/your-repo-name



EXPLORATORY ANALYSIS

- Loaded the wrangled dataset into a SQLite / Db2 table via %sql / ibm_db
- Queries covered: distinct launch sites, payload totals by customer, average payload by booster, first successful landing date, boosters meeting payload + outcome criteria, mission-outcome counts, max-payload boosters, month-level 2015 records, and a ranked landing-outcome count (2010–2017)
- Each query answers one specific analytical question used later in the results section

Query Categories Covered

Launch Sites

Payloads

Success Rates

Rankings

Time Analysis

Notebook / Code: github.com/your-username/your-repo-name



The screenshot shows a JupyterLab interface with a terminal window. The terminal has a light gray background and displays the following commands and output:

```
[16]: %load_ext sql

The sql extension is already loaded. To reload it, use:
      %reload_ext sql

[17]: import csv, sqlite3
import prettytable
prettytable.DEFAULT = 'DEFAULT'

con = sqlite3.connect("my_data1.db")
cur = con.cursor()

[18]: !pip install -q pandas

[19]: %sql sqlite:///my_data1.db

[20]: import pandas as pd
df = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/labs/module_2/data/Spacex.
df.to_sql("SPACEXTBL", con, if_exists='replace', index=False, method="multi")

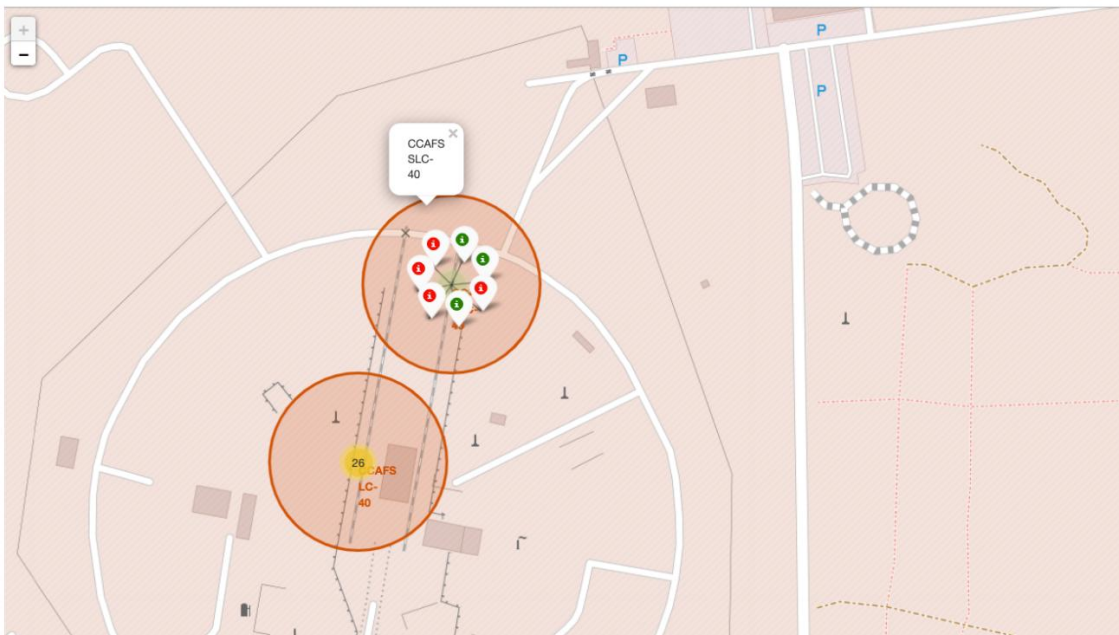
101
```




Interactive Visual Analytics



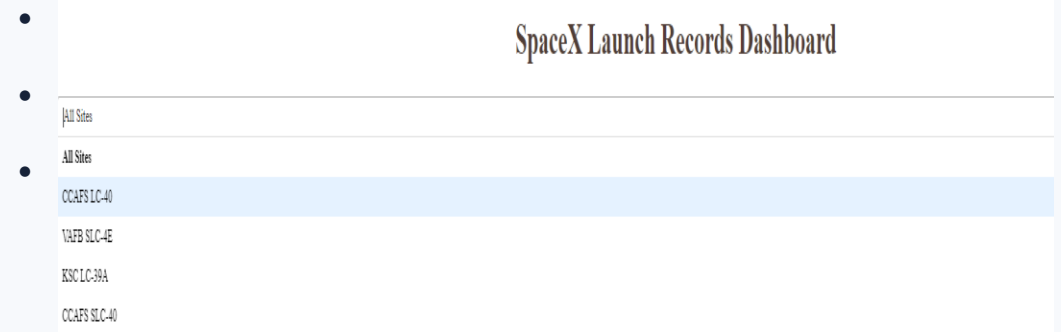
Folium Map



From the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates.



Plotly Dash App



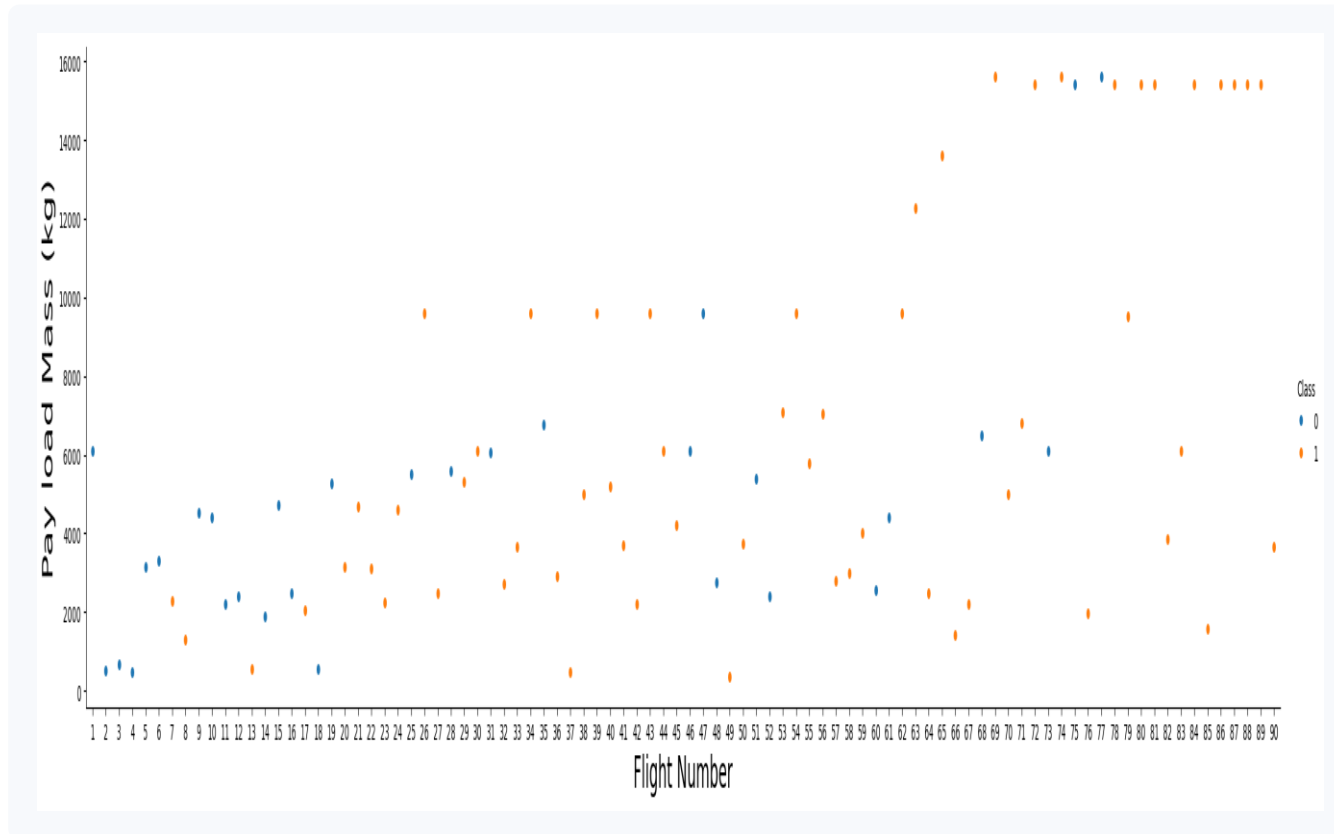
Notebook / App Code: github.com/your-username/your-repo-name



EDA RESULTS

Flight Number & Payload vs. Launch Outcome

Actual output from your edadataviz.ipynb notebook



Flight Number vs. Payload Mass, colored by outcome (Class)

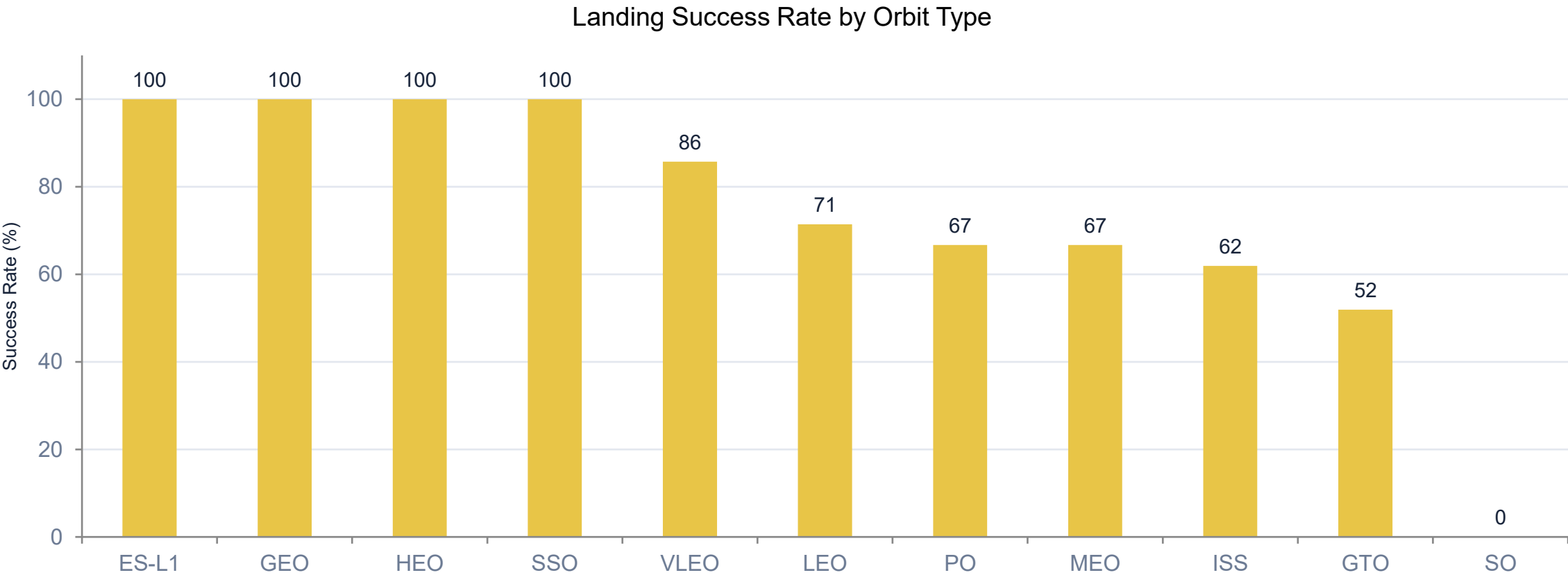


EDA Notebook: github.com/your-username/your-repo-name



Success Rate by Orbit Type

ES-L1, GEO, HEO and SSO orbits show a 100% landing success rate; GTO is lowest at 51.9%, reflecting its higher energy requirements.





Yearly Launch Success Trend

Success rate climbed from 0% before 2014 to 90% in 2019 as SpaceX iterated on Block 5 boosters and landing technique.





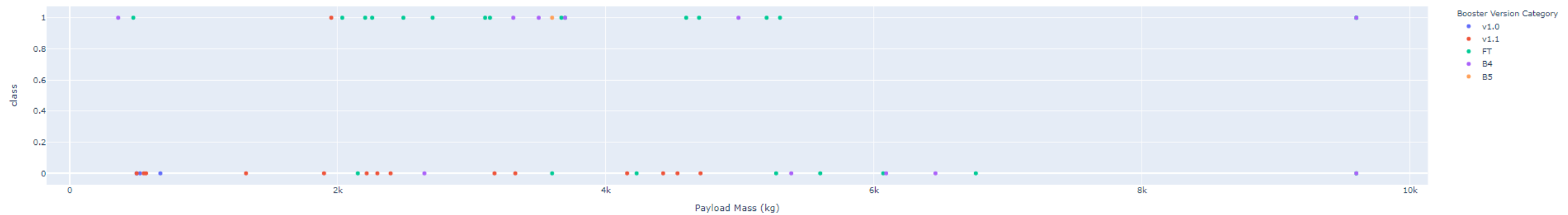
Launch Sites & Payload Totals

Query	Sample Result
Distinct launch site names	CCAFS SLC 40, KSC LC 39A, VAFB SLC 4E
Average payload mass by launch site (KSC LC 39A)	7,606.5 kg

Payload range (Kg):



Correlation between Payload and Success for all Sites



Computed directly from the 90-launch wrangled dataset (dataset_part_2.csv).



Success Rates & Rankings

Query	Sample Result
First successful ground-pad landing date	2015-12-22 (booster B1019)
Boosters: success (drone ship, ASDS) & payload 4,000–6,000 kg	B1021, B1022, B1026, B1031, B1046, B1059
Total successful vs. failed mission outcomes	Success: 60 Failure: 30
Boosters carrying the maximum payload mass	B1048, B1051 (15,600 kg each)

Computed directly from the 90-launch wrangled dataset (dataset_part_2.csv).



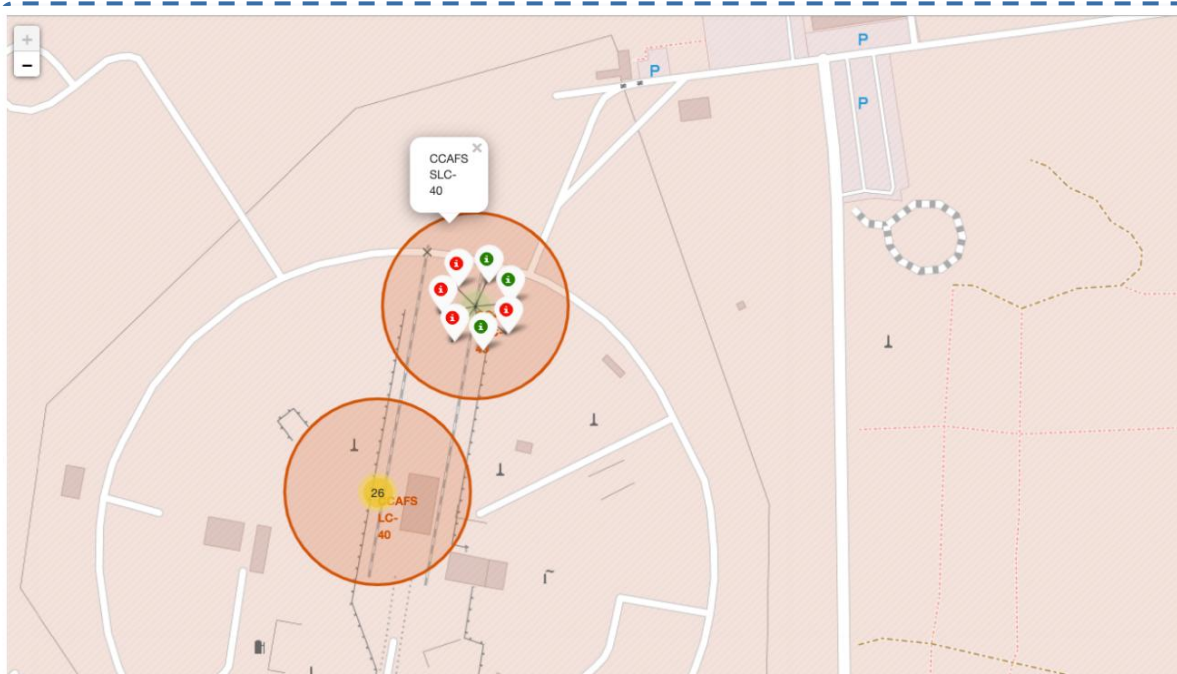
Time Analysis

Query	Sample Result
Month names for 2015 records with landing failure	January, April, June
Rank landing outcome counts across all 90 launches (2010–2020)	1) True ASDS — 41 2) No attempt — 19 3) True RTLS — 14 4) False ASDS — 6

Computed directly from the 90-launch wrangled dataset (dataset_part_2.csv).



Folium Map — Launch Site Markers



From the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates.

```
# Start location is NASA Johnson Space Center
nasa_coordinate = [29.559684888503615, -95.0830971930759]
site_map = folium.Map(location=nasa_coordinate, zoom_start=10)
```

We could use `folium.Circle` to add a highlighted circle area with a text label on a specific coordinate. For example,

```
# Create a blue circle at NASA Johnson Space Center's coordinate with a popup label showing its name
circle = folium.Circle(nasa_coordinate, radius=1000, color='#d35400', fill=True).add_child(folium.Popup('NASA JSC'))
# Create a blue circle at NASA Johnson Space Center's coordinate with a icon showing its name
marker = folium.map.Marker(
    nasa_coordinate,
    # Create an icon as a text label
    icon=DivIcon(
        icon_size=(20,20),
        icon_anchor=(0,0),
        html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' % 'NASA JSC',
    )
)
site_map.add_child(circle)
site_map.add_child(marker)
```



Folium Notebook: github.com/your-username/your-repo-name

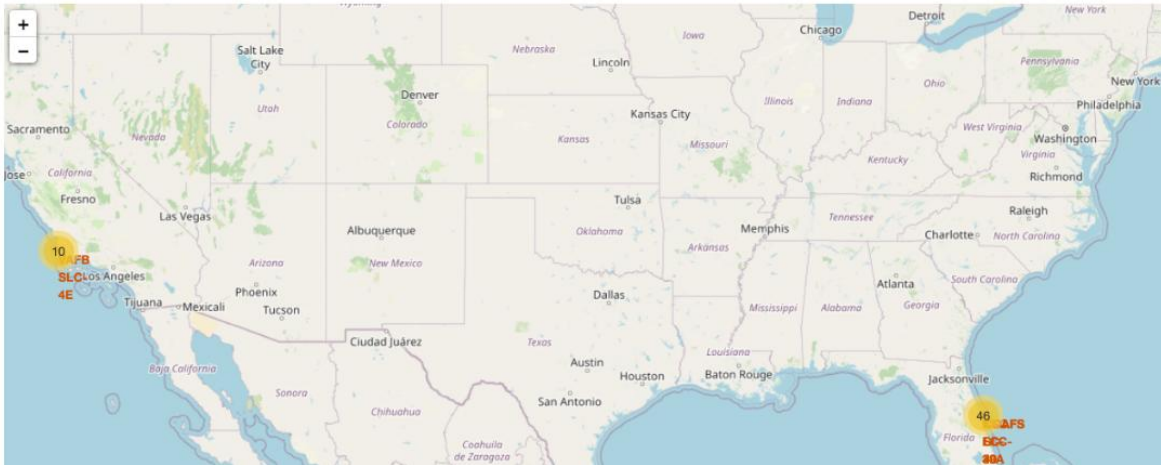


Folium Map — Launch Records & Proximities

```
# marker = folium.Marker(...)
marker_cluster.add_child(marker)
```

site_map

Your updated map may look like the following screenshots:



```
marker_cluster = MarkerCluster()
```

TODO: Create a new column in `spacex_df` dataframe called `marker_color` to store the marker colors based on the `class` value

```
# Apply a function to check the value of `class` column
# If class=1, marker_color value will be green
# If class=0, marker_color value will be red
```

TODO: For each launch result in `spacex_df` data frame, add a `folium.Marker` to `marker_cluster`

```
# Add marker_cluster to current site_map
site_map.add_child(marker_cluster)
```

```
# for each row in spacex_df data frame
# create a Marker object with its coordinate
# and customize the Marker's icon property to indicate if this launch was succeeded or failed,
# e.g., icon=folium.Icon(color='white', icon_color=row['marker_color'])
for index, record in spacex_df.iterrows():
    # TODO: Create and add a Marker cluster to the site map
    # marker = folium.Marker(...)
    marker_cluster.add_child(marker)
```

site_map



Folium Notebook: github.com/your-username/your-repo-name



Plotly Dash — Interactive Dashboard

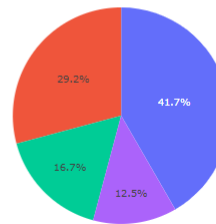
SpaceX Launch Records Dashboard

All Sites

X

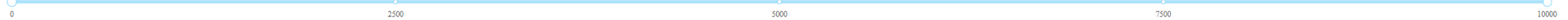


Total Success Launches By Site

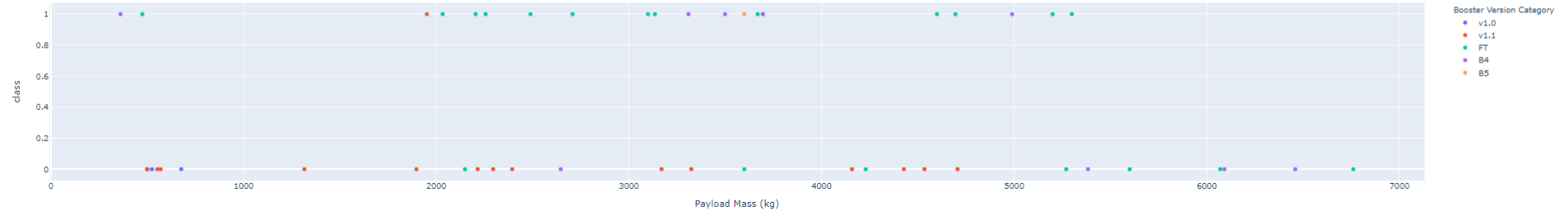


KSC LC-39A
CCAFS LC-40
VAFB SLC-4E
CCAFS SLC-40

Payload range (Kg):



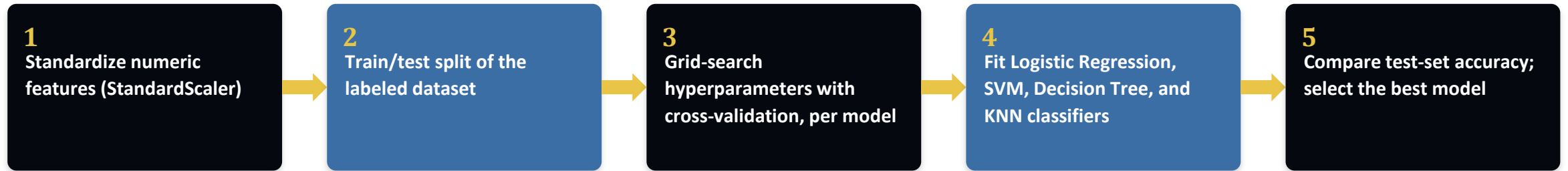
Correlation between Payload and Success for all Sites



Dash App Code: github.com/your-username/your-repo-name



Predictive Analysis Methodology



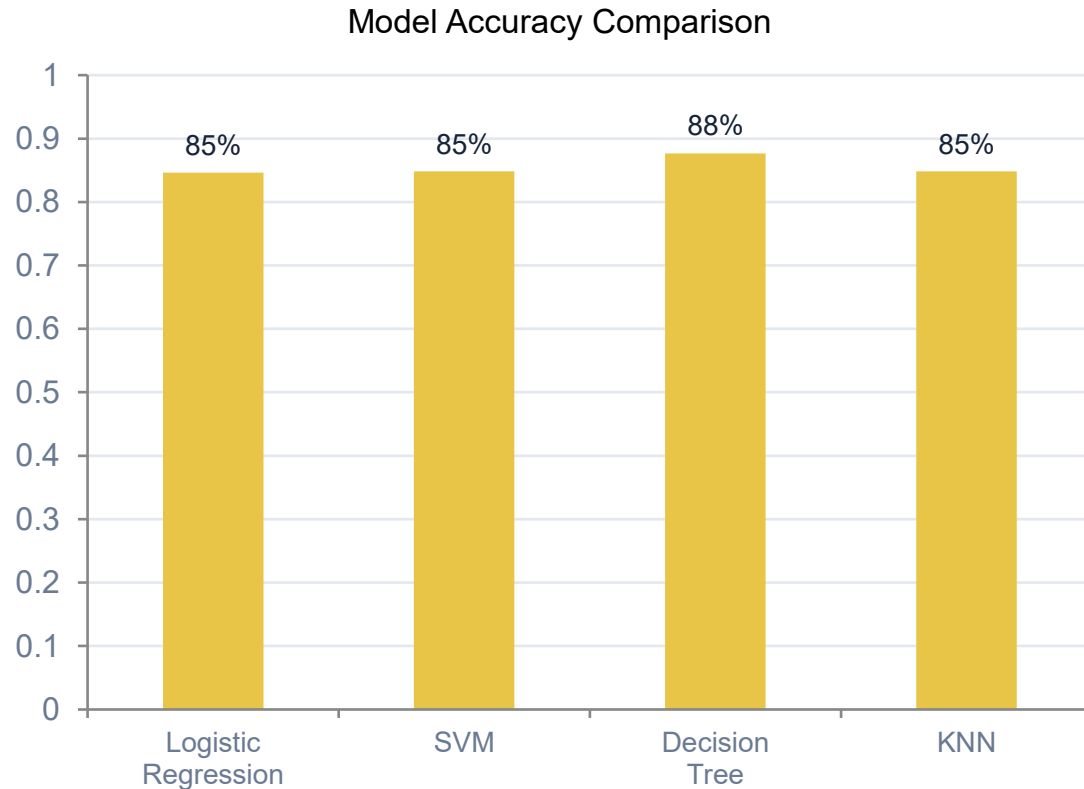
- Target variable: Class (1 = successful landing, 0 = failure), engineered during data wrangling
- Features: flight number, payload mass, orbit type, launch site, booster version/grid fins/legs, and reused-booster flag
- Each model tuned via GridSearchCV; best estimator per model type evaluated on the held-out test set



Modeling Notebook: github.com/your-username/your-repo-name



Model Comparison & Best-Model Results



Best Model: Decision Tree — 87.7% Accuracy

Best params: gini, depth=8, min_leaf=2, min_split=10, best splitter

Decision Tree scored best in cross-validation (87.7%), ahead of SVM/KNN (84.8%) and Logistic Regression (84.6%), and was selected as the final model.



Conclusion

- The best prediction model is the Decision Tree Classifier with 87% accuracy
- Best predictors of landing success: Payload Mass, Orbit, boosters, payload mass